Jose Velazquez Saenz

CSD380: DevOps

Professor Sue Sampson

Security Controls in Shared Source Code Repositories

Why Shared Repositories Need Strong Controls

- Source code reveals architecture, business logic, and internal implementation details.
- Hardcoded credentials can expose cloud accounts, APIs, databases, and deployment systems.
- Dependencies and build scripts can introduce supply-chain risk if changes are not reviewed.
- Auditability matters: teams need to know who changed what, when, and why.

Repository Security Goal

1. Prevent unauthorized access
2. Catch risky changes early
3. Keep secrets out of code
4. Maintain a traceable history

Control Access with Least Privilege

- Use role-based access controls so developers, reviewers, maintainers, and administrators have appropriate permissions.
- Require multi-factor authentication for repository accounts, especially accounts with administrative or release permissions.
- Review access regularly and revoke access for users who no longer need it.
- Avoid shared accounts because they make activity harder to trace and weaken accountability.

Protect Important Branches

01

Protect main, master, release, and production branches from direct pushes.

02

Require pull requests before merging changes into protected branches.

03

Require at least one qualified reviewer before approval, and require additional review for sensitive files.

04

Use status checks so tests and scans must pass before code can be merged.

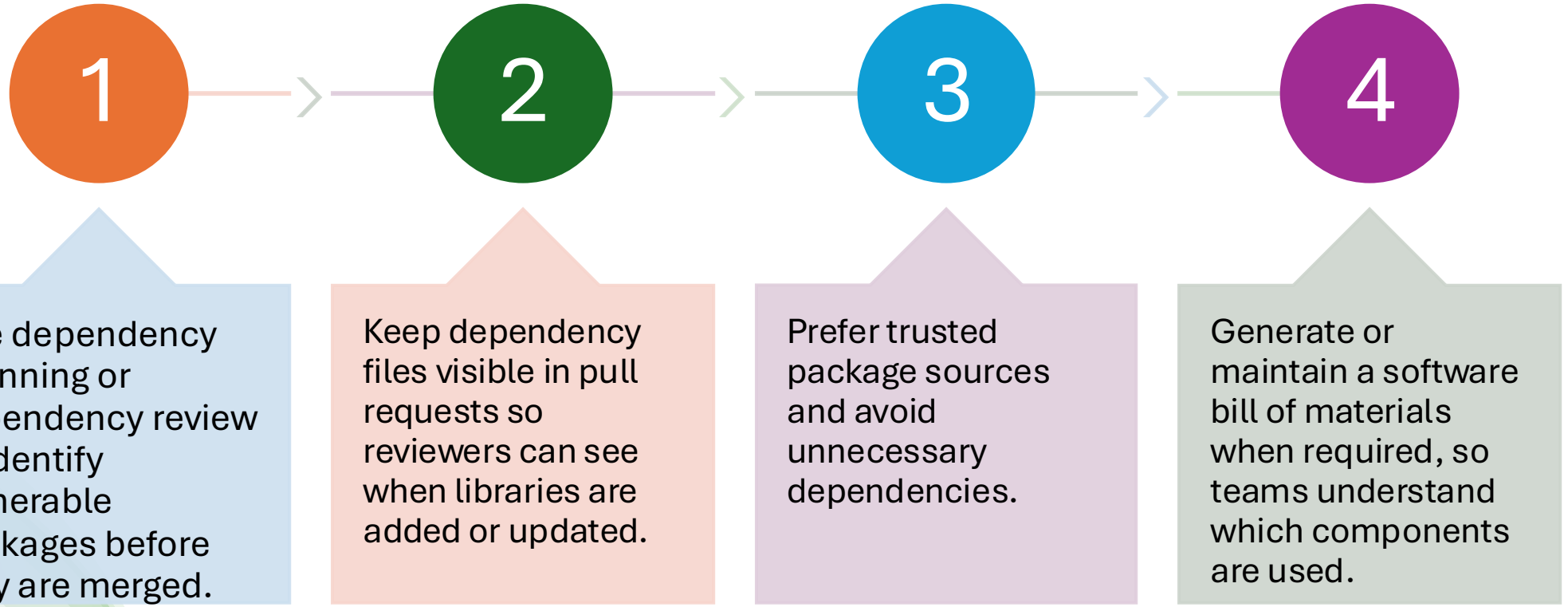
Require Peer Review and Code Review

- Peer review helps catch logic errors, insecure coding patterns, and accidental exposure of sensitive information.
- Code review should evaluate security, maintainability, and whether the change matches the intended requirement.
- OWASP recommends source code review to find vulnerabilities that automated tools may miss.
- Audit trails should also be used, as approvals are recorded in the change history.

Scan for Secrets and Sensitive Data

- Enable secret scanning to detect exposed API keys, tokens, passwords, and other sensitive data.
- Block or alert on commits that contain known secret patterns.
- Store credentials in approved secret managers or protected environment variables instead of repository files.
- When a secret is exposed, rotate it immediately, as deleting it from the repository does not remove the risk from previous commits.

Secure Dependencies and Supply Chain



Automate Security Checks in CI/CD

Run analysis, unit tests, dependency checks, and security scans as part of the build pipeline.

Use required checks to prevent risky changes from reaching protected branches or production deployments.

Automated controls should provide fast feedback so developers can fix issues early.

Manual approval should be reserved for higher-risk changes, not used as the only security control.

Logging, Monitoring, and Governance

Keep	Keep audit logs for pushes, pull requests, permission changes, branch rule changes, and administrative actions.
Monitor	Monitor unusual activity such as unexpected access, sudden permission changes, or suspicious token use.
Document	Document repository security policies so teams understand how changes should be reviewed and merged.
Test	Regularly test controls and update them as tools, threats, and team structures change.

References

Cornford, C., & Greiler, M. (2017, July). Owasp Code Review Guide. Owasp Foundation. https://owasp.org/www-project-code-review-guide/

github docs. (2026a). *About protected branches* . https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches

github docs. (2026b). Best practices for securing code in your supply chain. https://docs.github.com/en/code-security/supply-chain-security/end-to-end-supply-chain/securing-code

github docs. (2026c). *Secret scanning* . https://docs.github.com/en/code-security/concepts/secret-security/secret-scanning

OpenAI. (2026). ChatGPT (40 mimi). https://chat.openai.com/chat

Souppaya, M., Scarfone, K., & Dodson, D. (2022, February). *SP 800-218, Secure Software Development Framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities | CSRC*. NIST. https://csrc.nist.gov/pubs/sp/800/218/final

Wichers, D. (2026). Source code analysis tools . owasp foundation. https://owasp.org/www-community/Source_Code_Analysis_Tools